

CÁC ĐIỂM CẦN ÔN TẬP

Các từ viết tắt cần lưu ý

XML	Extensible markup language.
SAN	Storage area network
WSDL	Web Services Description Language
DFS	Distributed File System
NFS	Network File System
NIST	National Institute of Standards and Technology

CHƯƠNG I – TỔNG QUAN – INTRODUCTION

Several notions (ChatGPT/Grok)

- The term cloud comes from Network diagrams
- Standard cloud computing pricing model: Pay-per-usage
- **Elasticity:** refers to the ability of a cloud computing system to *automatically and dynamically allocate or release resources in response to changes in demand*. It is closely related to cloud scalability but focuses specifically on the automatic adjustment of resources without requiring manual intervention.

Example: A video streaming service might experience varying demand depending on the time of day or the type of content being watched. For instance, a live sports event or a new season of a popular series might cause a sudden spike in traffic. Elasticity ensures the service can handle these fluctuations, scaling out resources dynamically and scaling back when traffic returns to normal.

- **Scalability:** refers to the ability of a cloud-based system or application to handle a growing amount of work, or its potential to accommodate growth without compromising performance.

Vertical scaling involves adding more power (CPU, RAM, storage) to an existing machine or server.

Example: If a website is running on a cloud server with 4GB of RAM and 2 CPUs, vertical scaling would involve upgrading to a machine with 8GB of RAM and 4 CPUs.

Horizontal scaling involves adding more machines (servers or instances) to a system to distribute the load across multiple nodes (machines).

Example: If the website experiences increased traffic, instead of upgrading to a more powerful machine, the cloud platform could automatically add more virtual machines or containers to handle the increased load.

- Utility value provided by a cloud-based service: access from anywhere
- Service-level agreement (SLA) in cloud field: identifies the roles and responsibilities of both the customer and the cloud service provider
- Business driver for cloud computing: strategic flexibility, IT staff reduction, cost reduction. Cloud computing is the convergence of Highly scalable, Flexible and dynamic platform & Facilitating hardware access to many users
- *Capital expenses:* Capital expenditures (CapEx) are funds used by a company to acquire, upgrade, and maintain physical assets such as property, plants, buildings, technology, or equipment (server, etc).

- *Operating expenditures, or “OpEx,”* refers to the costs incurred by a business for its operational activities. In other words, operating expenses are the costs that a company must make to perform its operational activities.
- **Load balancing** is the process of distributing incoming network traffic or workload across multiple servers or resources to ensure no single server becomes overwhelmed, optimizing resource usage, improving performance, and ensuring high availability. In cloud environments, load balancing plays a crucial role in ensuring that applications, websites, and services remain responsive and reliable even under heavy traffic.

DEPLOYMENT MODEL

Private cloud

A private cloud is a computing environment dedicated to a single organization, offering cloud-like benefits (e.g., scalability, self-service) but with greater control, security, and customization. Unlike public clouds (e.g., AWS, Azure), which share resources across multiple customers, a private cloud is isolated—either hosted on-premises or by a third-party provider. Services can only be accessed by a restricted number of people.

Several use cases:

- Enterprise IT: Running legacy apps that don’t work well on public cloud.
- Data Sensitivity: Banks securing customer data or hospitals protecting patient records.
- Performance-critical Apps: Manufacturing systems needing low latency.

Public cloud

Public cloud refers to a computing model where cloud services—such as storage, computing power, or software—are provided by third-party vendors over the internet to multiple users or organizations. These services are hosted on the provider's infrastructure, typically in large, shared data centers, and are accessible to anyone who purchases them, making them "public" as opposed to private, on-premises systems. Major players include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP), which collectively dominate the market.

Hybrid cloud

Hybrid cloud is a computing model that combines public cloud and private cloud (or on-premises infrastructure) into a single, integrated system. It allows data, applications, and workloads to move seamlessly between the two environments, leveraging the strengths of both while addressing their limitations. Hybrid cloud *offers a mix of public and private cloud features.*

Example use case:

- A bank might use AWS’s public cloud for customer mobile apps, but process transactions on a private cloud to meet regulatory standards, with real-time data syncing between them.
- An organization that wants to leverage its existing IT infrastructure but has occasional periods of high demand

Multi-cloud

Multi-cloud refers to the use of multiple cloud computing services from different providers (e.g., AWS, Microsoft Azure, Google Cloud) within a single organization or system. This approach can enhance flexibility, avoid vendor lock-in, and optimize costs or performance by leveraging the strengths of various platforms.

Use Case: E-Commerce Platform with Seasonal Traffic Spikes

Imagine an e-commerce company that experiences significant traffic surges during holiday seasons, like Black Friday or 11.11/12.12. To handle this efficiently, they adopt a multi-cloud strategy:

- **Primary Operations on AWS:** The company uses Amazon Web Services (AWS) for its core infrastructure—hosting its website, managing customer databases, and running its primary application. AWS's robust EC2 instances and RDS (Relational Database Service) handle the day-to-day load reliably.
- **Burst Capacity on Google Cloud:** During peak shopping seasons, the company leverages Google Cloud Platform (GCP) for additional compute power.
- **Analytics on Microsoft Azure:** The company also uses Azure for its data analytics and machine learning needs. Customer purchase data and browsing patterns are fed into Azure's Synapse Analytics and AI tools to generate real-time insights, helping the marketing team adjust promotions on the fly.

CLOUD SERVICE MODEL S

Infrastructure as a Service (IaaS)

IaaS, or Infrastructure as a Service, is a cloud computing model where a third-party provider delivers virtualized computing resources over the internet. Think of it as renting the foundational building blocks of IT—servers, storage, and networking—without the hassle of owning and maintaining physical hardware. *IaaS allows the greatest flexibility for application development* (IaaS provides greater flexibility in terms of configuring and customizing the infrastructure, making it suitable for applications with specific requirements and customizations.)

Examples of IaaS Providers:

- Amazon Web Services (AWS): EC2, S3, VPC.
- Microsoft Azure: Virtual Machines, Blob Storage, Virtual Network.
- Google Cloud Platform (GCP): Compute Engine, Cloud Storage.
- Others: IBM Cloud, Oracle Cloud, DigitalOcean.

Example use case:

A startup might use AWS EC2 to host its app, S3 for user-uploaded files, and a load balancer to distribute traffic—all without buying a single server.

Platform as a Service (PaaS)

PaaS, or Platform as a Service, is a cloud computing model where a provider delivers a platform—hardware, operating systems, and development tools—over the internet, *allowing users to build, deploy, and manage applications without worrying about the underlying infrastructure*. It sits between IaaS (raw infrastructure) and SaaS (ready-to-use software) in the cloud stack, offering a middle ground for developers and businesses.

Examples of PaaS Providers:

- Google App Engine: Flexible app hosting with auto-scaling.
- Microsoft Azure App Services: Web apps, APIs, and mobile backends.
- AWS Elastic Beanstalk: Simplified app deployment with AWS integration.
- Others: Heroku, Red Hat OpenShift, IBM Cloud Foundry.

Example use case:

A developer builds a web app on AWS, using its built-in PostgreSQL database and auto-scaling. They deploy in hours, not days, and don't touch a single server.

Software as a Service (SaaS)

SaaS, or Software as a Service, is a cloud computing model where software applications are delivered over the internet by a third-party provider. Users access these apps—typically through a web browser—without installing or maintaining them locally. It's the most user-facing layer of the cloud stack, sitting above IaaS (infrastructure) and PaaS (platform), and it's designed for ease of use rather than technical control.

Examples of SaaS Providers:

- Google Workspace: Gmail, Docs, Drive.
- Microsoft 365: Word, Excel, Teams.
- Salesforce: CRM for sales and customer management.
- Others: Dropbox, Zoom, Slack, Adobe Creative Cloud.

Example use case:

A student team uses Canva (SaaS) to design graphics. They log in, create, and collaborate in real-time—no software downloads, no server management.

Tham khảo:

- https://aws.amazon.com/documentation-overview/?nc2=h_ql_doc_do
- <https://cloud.google.com/learn/training?hl=en>
- <https://learn.microsoft.com/en-us/training/azure/>
- <https://www.geeksforgeeks.org/cloud-computing/>
- <https://www.ibm.com/think/topics/cloud-computing>
- ChatGPT/Grok3

CHƯƠNG II – ẢO HÓA

Several notions (ChatGPT/Grok)

- *Isolating:* Virtualization *creates isolated environments (virtual machines, or VMs) on a single physical machine, ensuring each VM operates independently*, preventing interference or crashes between them or the host system, a crucial aspect of security and stability. *Applications are installed and run on the guest OS inside the VM. Each VM operates in isolation, ensuring software dependencies do not interfere across VMs. In data centers, VMs enhanced security through hardware isolation.*
- A Virtual Machine (VM): is a software-defined computer, or an *emulation of a physical computer*, that uses resources like CPU, memory, and storage from a physical machine to run its own operating system and applications in isolation. *Primary benefit of virtualization in cloud computing: resource optimization/ enabling efficient resource allocation and cost savings.*
- Portability: VM (Virtual Machine) portability refers to *the ability to move a virtual machine, including its configuration and operating system, from one physical or virtual machine (host) to another* without significant changes or rebuilding
- *Snapshots:* A virtual machine (VM) snapshot captures the state and data of a VM at a specific point in time, like a "point-in-time copy" or "picture", allowing you to revert the VM to that state if needed.
Example: You can take a snapshot before installing a new application or making significant configuration changes, then revert to the snapshot if something goes wrong.
- Virtual disk manager: a utility that allows you to create, manage, and modify virtual disk/ virtual disk files.
- Server consolidation: Server consolidation in cloud computing refers to the process of combining multiple servers into a single, more powerful server or cluster of servers. Server consolidation is typically achieved through the use of virtualization technology, which allows multiple virtual servers to run on a single physical server. This allows for better utilization of resources, as well as improved scalability and flexibility. It also allows organizations to reduce the number of physical servers they need to maintain, which can lead to *cost savings on hardware, power, and cooling (energy consumption).* *Server consolidation through virtualization is enabled by using hypervisor.*
- Host system: Host system virtualization involves running one or more virtual machines (VMs), each with its own operating system, on a single physical machine called the host, leveraging a hypervisor to manage resources. Host system has *main operating system running on a physical machine*
- A "guest operating system" (guest OS): is *an operating system* (like Windows, Linux, or macOS) *that runs within a virtual machine (VM)* or on a partitioned disk, managed by a hypervisor and operates independently of the host OS.

Guest OS:
Operating system running in a virtual machine (VM)

Guest Machine:
Virtual machine created through virtualization

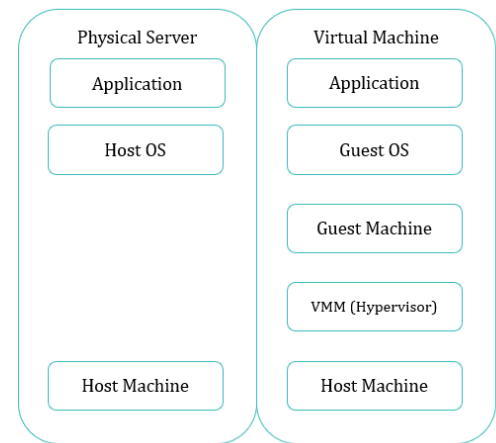
Hypervisor:
Virtualization software layer, or Virtual Machine Manager (VMM)

Host OS:
Operating system running in a physical machine

Host Machine:
Physical machine

A **host machine** is a physical computer that can run multiple VMs, and an OS installed and running on the host machine is a **host OS**. VMs running on a host machine are called **guest machines**. The OS installed on VMs is called a **guest OS**.

The core of virtualization technology between **the host OS** and **guest OS** is a **hypervisor**, which is sometimes called **Virtual Machine Manager (VMM)**.



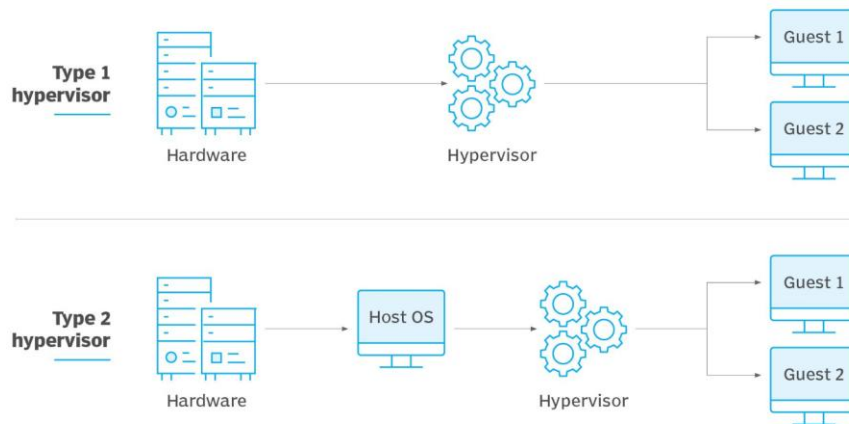
- **Xen Hypervisor:** Xen is an open-source hypervisor that enables virtualization. It allows multiple virtual machines (VMs) to run on a single physical machine, utilizing hardware resources in an efficient manner. Originally developed by the University of Cambridge, Xen is one of the most popular and widely used hypervisors, especially for server and cloud environments. *Example: Amazon EC2* (Elastic Compute Cloud) was one of the early adopters of Xen for its cloud-based virtualization.

Hypervisor

Virtualization requires the use of a hypervisor, which was originally called a virtual machine monitor or VMM. A hypervisor abstracts operating systems and applications from their underlying hardware. *A hypervisor manages the virtual machine and hardware interaction. Hypervisor dynamically allocates hardware resources to virtual machines based on their demands.*

There are two types of hypervisors: Type 1 and Type 2 hypervisors. Both hypervisor varieties can virtualize common elements such as CPU, memory and networking. But based on its location in the stack, the hypervisor virtualizes these elements differently.

The Type 1 hypervisor sits on top of the bare metal server and has *direct access to the hardware resources without an underlying operating system*. Because of this, the Type 1 hypervisor is also known as a *bare metal hypervisor*. In contrast, the Type 2 hypervisor is an application installed on the host operating system. It's also known as a *hosted or embedded hypervisor*.



Tham khảo:

- <https://aws.amazon.com/what-is/hypervisor/#:~:text=A%20hypervisor%20is%20a%20software,individual%20virtual%20machines%20as%20required.>
- <https://www.ibm.com/think/topics/hypervisors>
- <https://www.techtarget.com/searchitoperations/definition/hypervisor>

Ảo hóa hoàn toàn (Full virtualization) & Ảo hóa bán phần - ảo hóa song song (Paravirtualization)

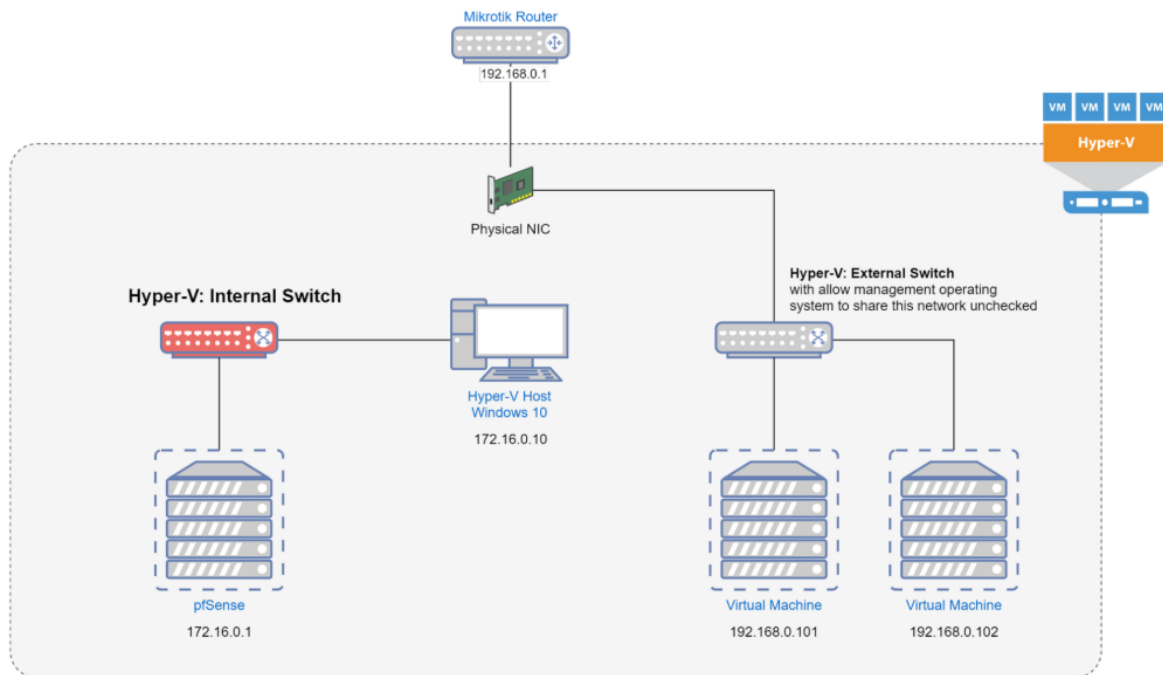
Full virtualization is a type of virtualization technology where an *entire physical computer's hardware environment—CPU, memory, storage, and peripherals—is simulated by a hypervisor* (a software layer). This allows a guest operating system (OS) to run on a virtual machine (VM) as if it were running directly on physical hardware, without needing any modifications to the guest OS. It's like creating a complete, isolated computer within another computer.

The hypervisor (e.g., VMware ESXi, Microsoft Hyper-V, or KVM) intercepts and emulates hardware calls from the guest OS, translating them to the actual physical hardware. This abstraction provides flexibility but can introduce some performance overhead due to the emulation.

Paravirtualization is a virtualization technique where the guest operating systems are aware they're running in a virtualized environment and communicate directly with the hypervisor. Unlike full virtualization, which emulates an entire hardware environment, paravirtualization requires the guest OS to be modified to work with the hypervisor. *Microsoft Hyper-V supports paravirtualization*

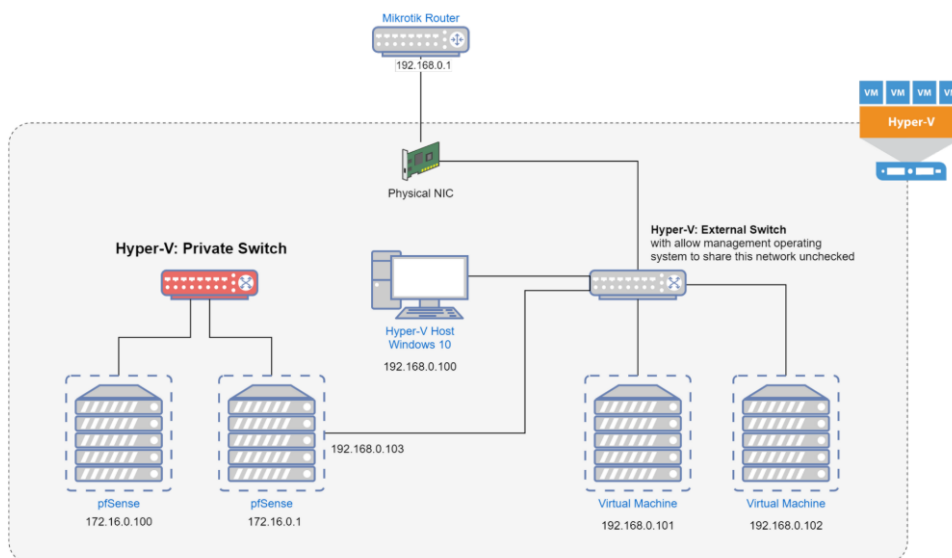
Virtual switch

A virtual switch, or vSwitch, is a software-based network switch that emulates the functionality of a physical switch, allowing virtual machines (VMs) to communicate within a virtualized environment and with external networks.



Types of Virtual Switches:

- **External:** Connects VMs to the physical network, allowing access to external resources.
- **Internal:** Facilitates communication only between VMs and the host server (Hyper-V host).
- **Private:** Used for communication solely between VMs, offering a secure and isolated network.



CHƯƠNG III – DATA & STORAGE

Several notions

- Benefit of using cloud computing for data storage: Improved data security due to centralized storage and advanced encryption; reduced costs compared to traditional on-premises data storage solutions; enhanced data accessibility from anywhere with an Internet connection
- SOAP
- **Managed Cloud Storage** refers to a cloud-based service that provides users with storage capacity managed and maintained by a third-party provider. This service typically includes features like data backup, file synchronization, scalability, and security, all handled by the service provider, reducing the need for users to manage these aspects themselves.

Examples of Managed Cloud Storage Providers:

- *Amazon Web Services (AWS) S3: Scalable object storage service with high durability and security.*
- Google Cloud Storage: Provides scalable and secure storage with high performance.
- Microsoft OneDrive for Business: Cloud storage that integrates tightly with the Microsoft 365 ecosystem.
- Dropbox Business: Known for its simplicity and collaboration features.
- **Unmanaged Cloud Storage** refers to cloud storage services where users are responsible for managing their own data, access controls, and some aspects of security, unlike managed cloud storage where the provider takes care of these tasks. While the core concept is similar to managed cloud storage in that it provides online data storage, the key difference lies in the level of management and support provided by the cloud service provider. With unmanaged cloud storage, users have more control but also more responsibility.
- For a cloud backup storage solution, **AES-256** is typically considered the **gold standard** for encryption, ensuring the highest level of protection for your data.
- Shadow data are usually stored in unmanaged data sources.
- Cloud storage gateways: is a technology that provides a *bridge between an organization's on-premises data storage systems and a cloud storage service*. It allows businesses to integrate and use cloud storage without requiring significant changes to their existing infrastructure. Essentially, it enables organizations to access cloud storage as if it were part of their local environment, offering seamless connectivity between on-premises data and the cloud.

Block, object & file storage

	Object storage	Block storage	File storage
--	----------------	---------------	--------------

Type of storage	Objects stored in scalable buckets	Fixed-size blocks in a rigid arrangement	Files organized hierarchically in folders and directories
Volume of data	Supports high data volumes	Supports high data volume	Better for lower volumes of data
Data management	Custom metadata provides easy searchability	More limited search and analytics capabilities	Hierarchical structure works well for simpler, smaller datasets
Cost	Pay-as-you-go pricing, more cost-effective	More costly, storage purchased as fixed blocks of storage	More costly, requires purchasing new storage devices to scale out
Performance	Slower performance, longer processing times	Super low latency and high performance	Performance impacted by higher data volume
Scalability	Highly scalable	Limited scalability	Limited scalability
Ideal for	Big data storage, static unstructured data, analytics, rich media files, and backups	Transactional, structured data, storage for databases, disks for VMs, and caching	Shared file storage, unstructured data

Source: <https://cloud.google.com/discover/object-vs-block-vs-file-storage?hl=en>

Block storage, sometimes referred to as block-level storage, is a technology that is used to store data files on storage area networks (SANs) or cloud-based storage environments. Developers favor block storage for computing situations *where they require fast, efficient and reliable data transportation*. Block storage breaks up data into blocks and then stores those blocks as separate pieces, each with a unique identifier. The SAN places those blocks of data wherever it is most efficient. That means it can store those blocks across different systems and each block can be configured (or partitioned) to work with different operating systems. *Block storage usually imposes additional overhead on clients and offer faster transfer*

File Storage refers to the method of storing digital files on a storage medium that can be accessed, retrieved, and managed. It is one of the primary forms of data storage used in computing environments, especially in the context of data management, organization, and retrieval. File storage systems are designed to store files (such as documents, images, videos, and application data) in a structured and organized manner, making it easier to access, back up, and manage. File storage can be local (on physical devices) or remote (cloud-based), depending on the infrastructure and use case. There are different types of file storage systems, each offering unique features based on performance, scalability, and management capabilities.

Object Storage is a data storage architecture that stores data as objects rather than in files or blocks. Each object typically consists of the data itself, metadata (which describes the data),

and a unique identifier (often referred to as an object ID). This model is highly scalable, flexible, and efficient for managing large volumes of unstructured data like media files, backups, archives, and big data analytics.

In contrast to traditional file storage (which organizes data into a hierarchical file system), object storage does not use a directory structure but rather addresses each object with a unique identifier. This makes it particularly suitable for storing vast amounts of unstructured data, which is often the case in cloud environments.

Tham khảo:

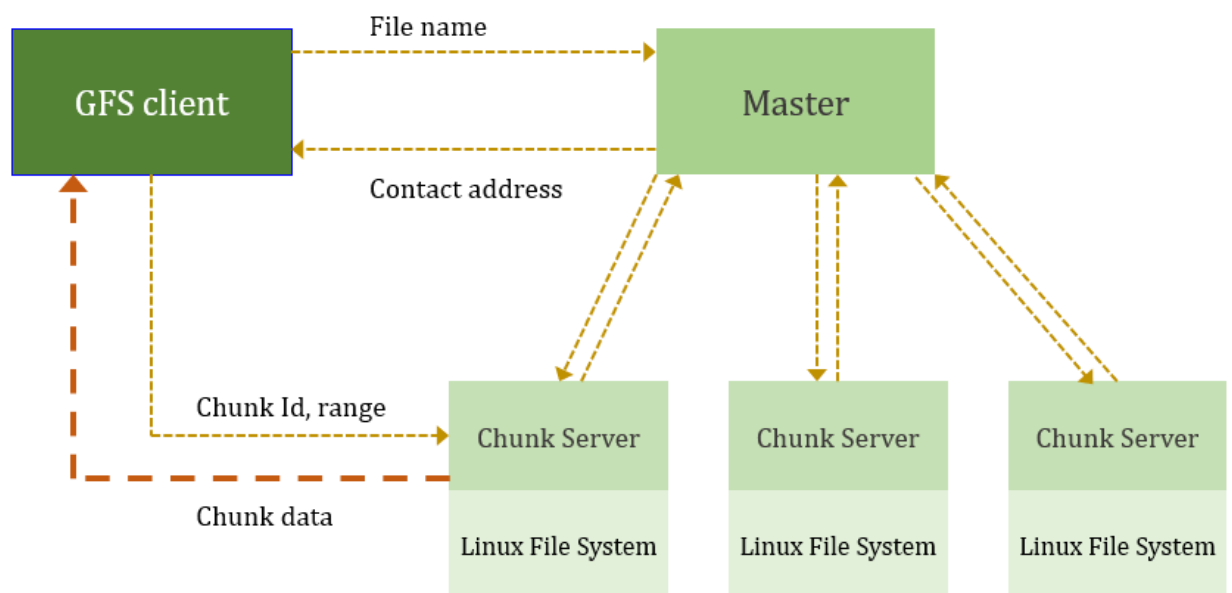
- <https://viettelidc.com.vn/tin-tuc/so-sanh-file-storage-block-storage-va-object-storage>
- <https://aws.amazon.com/compare/the-difference-between-block-file-object-storage/>
- <https://cloud.google.com/discover/object-vs-block-vs-file-storage?hl=en>
- <https://www.ibm.com/think/topics/object-vs-file-vs-block-storage>

Google File System

The Google File System (GFS) is a highly scalable, distributed file system developed by Google to meet the demands of large-scale data storage and processing. It was specifically designed to support the needs of Google's data-intensive applications and infrastructure, which require handling massive amounts of data reliably and efficiently.

Master-Slave Architecture:

- GFS follows a master-slave architecture, where the master node maintains metadata (like file names, locations of data blocks, and file system structures) and the chunk servers store actual data blocks.
- The master node doesn't store file data itself but holds the metadata about files, while chunk servers manage the data. The master node coordinates the system and ensures consistency.



Tham khảo:

- <https://static.googleusercontent.com/media/research.google.com/en//archive/gfs-sosp2003.pdf>
- <https://brabalawuka.cc/posts/study/gfs/>

MAP REDUCE

MapReduce is a programming model and an associated processing technique *used for handling and processing large datasets in parallel across a distributed cluster of computers*.

Development of parallel-programming model needs to handle the tasks like scheduling, intra-cluster communication and task monitoring by the developer. But, by using the library routines of MapReduce programming model, the developers can create parallel-programming effortlessly without worrying about all of these issues.

MapReduce is a framework which splits the chunk of data, sorts the map outputs and input to reduce tasks. The file-system stores the work and input of jobs. Re-execution of failed tasks, scheduling them, and monitoring them is the task of the framework. The architecture of MapReduce basically has two main processing stages, Map and Reduce. The MapReduce happens in Job tracker while the intermediate processes will take place in between the Map and Reduce phases. The sort and shuffle are the tasks taken up by Map and Reduce, which are done intermediately. The local file system stores the intermediate data.

MapReduce is Data processing layer of Hadoop

Map Phase:

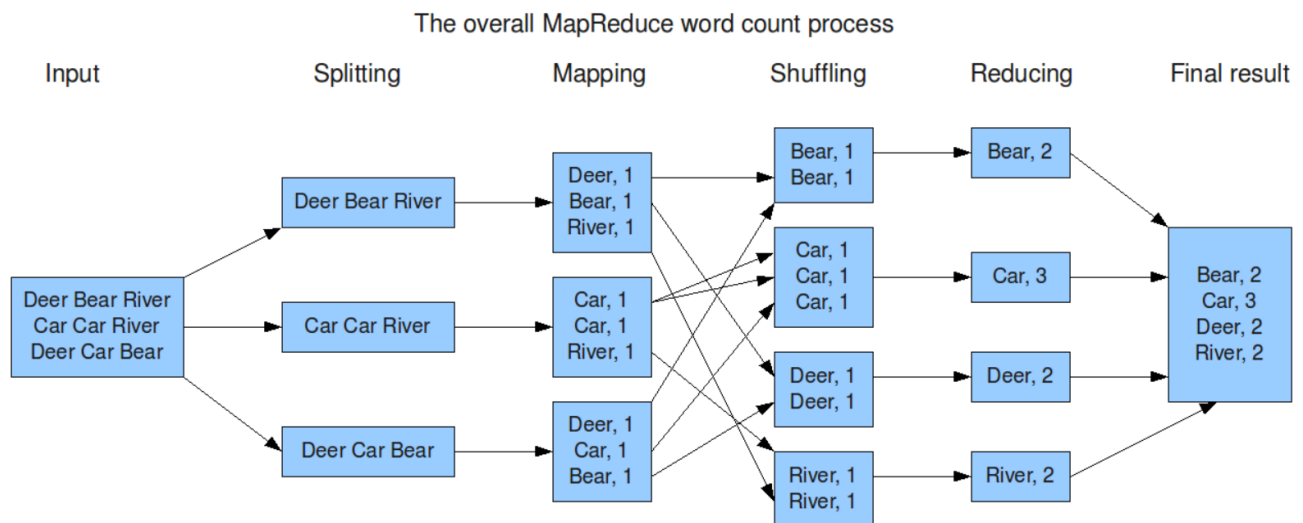
- Function: The map function takes input data, processes it, and produces a set of intermediate key-value pairs.
- Each mapper works on a portion of the data in parallel. It reads input data (often stored in a distributed file system like HDFS), processes it, and then outputs key-value pairs that represent some form of transformation of the original data.
- For example, if the input is a large collection of text documents, the map function could break each document into words and count their occurrences.
- *Number of maps is usually driven by the total size of inputs*

Shuffle and Sort Phase *(occur simultaneously)*:

- Function: After the map phase, the system performs a shuffle and sort operation to group the intermediate key-value pairs by their keys.
- This phase redistributes the intermediate key-value pairs so that all values associated with the same key are sent to the same reducer. This ensures that the reduce phase works on grouped data.
- This is an internal step that typically involves network communication.

Reduce Phase:

- Function: The reduce function processes each group of intermediate key-value pairs. It aggregates or combines the values corresponding to each key to produce a final result.
- For example, if the map function produced word counts, the reduce function would sum up the counts for each word.
- The reduce function produces the final output of the MapReduce job.



The figure shows the execution of the word count program. First, the input is split line by line and then, each line goes to a different mapper in which, for each word in the line, it emits a tuple containing the word (the key) and a counter (the value) starting with 1. In the next stage, the tuples are shuffled (by key) into a logical order (in this case in alphabetical order). The goal of shuffling is to have all the tuples with a same key going to the same reducer. The reducers receive, for each key, all the values present in the tuples and, in the word count example, they simply sum all the values for a key and emit a resulting tuple consisting of the key and the sum, which is the number of occurrences of that key.

Tham khảo:

- <https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>
- <https://www.ibm.com/docs/en/netezza?topic=started-wordcount-example>

HADOOP

Several notions

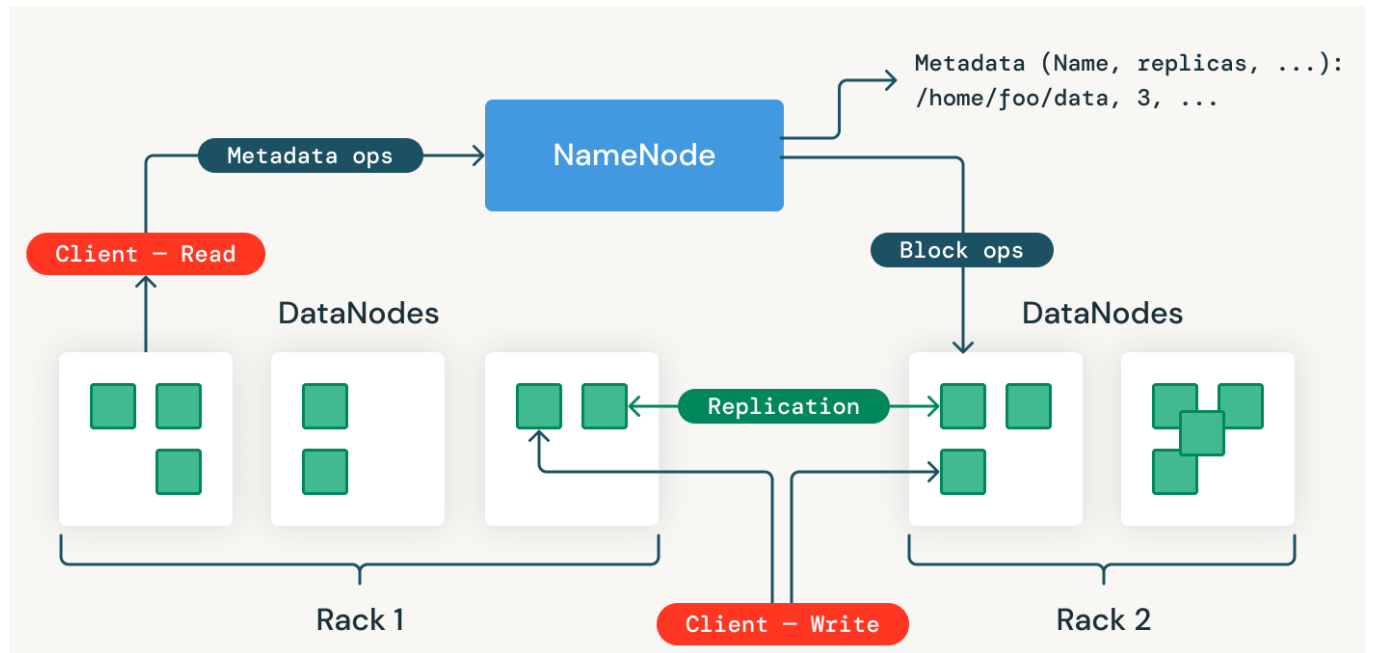
- The operation (or on Hadoop: edit) log is essential to ensure that the NameNode can recover after a crash.

- The default heartbeat interval for a Hadoop DataNode is 3 seconds
- The **replication factor** in the **Hadoop Distributed File System (HDFS)** refers to the number of copies of each data block stored across the cluster. It is an important parameter that helps ensure data reliability, fault tolerance, and availability within the system. The default replication factor in HDFS is **3**. This means that each block of data is replicated three times across different DataNodes in the Hadoop cluster.
- The purpose of the setup/cleanup methods in a Hadoop job is to enable both mappers and reducers to initialize a global resource.

In Hadoop, NameNode and DataNode are core components of the Hadoop Distributed File System (HDFS), and they work together to handle data storage, retrieval, and management in the Hadoop ecosystem.

Hadoop Architecture Overview

- NameNode:
 - The **NameNode** is the master node that manages the **metadata** for the HDFS. It keeps track of the structure of the filesystem, the mapping of files to blocks, and the location of blocks across the DataNodes.
 - *It does not store the actual data (the content of files); instead, it stores metadata such as:*
 - File names
 - Directories
 - Permissions
 - Block locations (which DataNode holds which blocks)
- DataNode:
 - *The **DataNode** is the worker node responsible for storing the actual data in the HDFS.*
 - Each DataNode manages its local storage and is responsible for serving data requests, such as read and write operations.
 - DataNodes regularly send **heartbeats** to the NameNode to confirm they are alive and functioning.
 - *A client reading a block of data from the cluster require no communication with the NameNode*



Tham khảo:

- <https://www.cl.cam.ac.uk/teaching/1920/CloudComp/lectures/storage-systems.pdf>

CHƯƠNG IV – Cloud Architecture

Several notions

- *A runtime environment in cloud computing provides the necessary infrastructure, libraries, and dependencies needed for an application or service to run and execute. It acts as a container for code execution, ensuring the code runs consistently across different systems.*
- *The cloud control plane is the management layer of a cloud environment. It provides the tools and interfaces (e.g., APIs, web consoles, etc) for: Provisions cloud resources (e.g., creating VMs, databases, containers), Manages the lifecycle of these resources (start, stop, scale, or delete), Provides APIs, dashboards, and automation for cloud services.*
- *Event-driven architecture: An event-driven architecture consists of event producers that generate a stream of events, event consumers that listen for these events, and event channels that transfer events from producers to consumers. Events are delivered in near real time, so consumers can respond immediately to events as they occur. Producers are decoupled from consumers: A producer doesn't know which consumers are listening. Consumers are also decoupled from each other, and every consumer sees all of the events.*

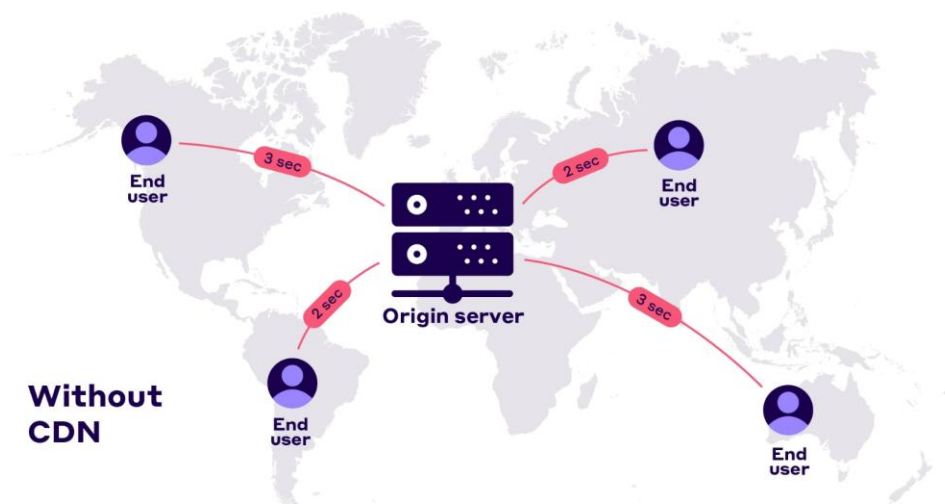
INFRASTRUCTURE

Load balancers

Load balancing is a common networking technique for distributing traffic among numerous servers in a server farm. Load balancers increase the availability and responsiveness of applications while also preventing server overload. Each load balancer lies between client devices and backend servers, accepting and then distributing incoming requests to any server that can handle them.

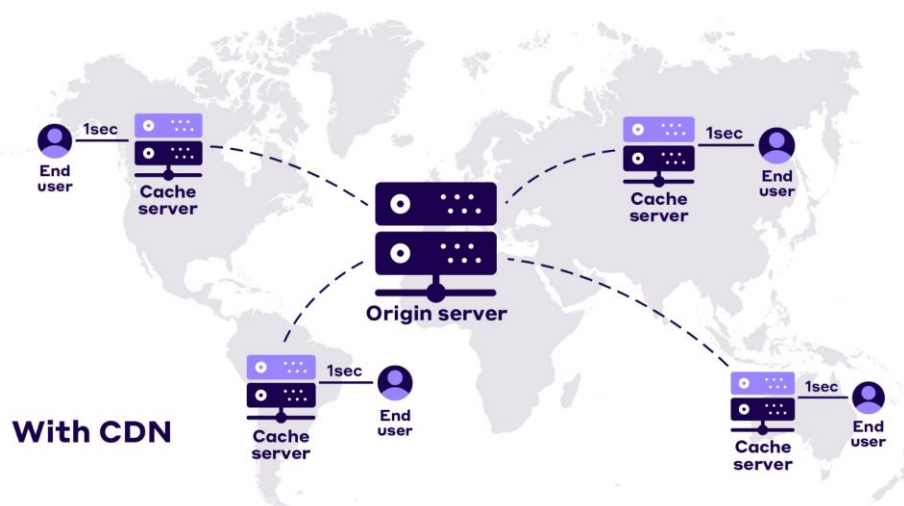
Content delivery networks

The problem of delivering content in cloud exists due to the distance between the source of the content and the locations of content consumers. In most cases, data centers are the originators of digital content. Cloud service providers set up a limited number of data centers (infrastructure) around the world and store all content in those data centers. Any request from a user for some content may travel a long distance over the network depending on the geographical distance between the data center and the user. Transferring huge content like the high-definition video files far across the network path to the user may cause performance issue.



To serve the purpose of instant delivery of content via network to the consumers, technologists have introduced the *concept of a dedicated network for faster content delivery*. Content generators like cloud service providers can make use of these networks for efficiently delivering their content to millions of consumers across the globe.

The basic idea is that instead of accessing data content in cloud centrally stored in a few data centers, it is better to replicate the instances of the data at different locations.



CDN enables faster delivery of content by caching and replicating content from 'content source' to multiple 'edge servers' or 'cache servers' which are strategically placed around the globe.

In cloud computing, the users *avail computing resources and perform all of the computation tasks generally via Internet*. Data-intensive computation tasks *require high-speed availability of data so that users can experience on-premises like performances*. Thus, every cloud service provider needs to *use content delivery network service in the background to deliver speedy and high-performing content to consumers*.

CLOUD MIGRATION

Cloud migration refers to the migration or movement of applications, data, services, or any proprietary facility from an organization's onsite computing facility to the cloud, which provides a cost-effective, reliable, versatile, interoperable, flexible, and manageable environment. Cloud migration is a systematic process that includes many phases. Moreover, no single migration strategy or method is common to all applications. The method chosen depends on the nature or type of application. It's important for any enterprise to carefully analyze its motivations for migration from a business, technical, and security and privacy viewpoint.

Typical Examples Where Cloud Computing Provides Benefits

- Applications experiencing huge traffic;
- Applications experiencing uncertain loads;
- Applications that need quick and continuous development and deployment;
- Applications that have growing storage demands;

- Applications experiencing peak loads as a result of seasonal traffic;
- Applications that have no sensitive data;
- Enterprises needing to distribute an application across different locations;
- Enterprises needing to undergo frequent software upgrades, regular database backup, and periodic maintenance

While performing the opportunity evaluation phase for cloud migration an enterprise decides whether to opt for the cloud or not based on an analysis of the relevant business case. Before an enterprise opts for cloud migration the type of each application is identified and the application thoroughly analyzed to see how it can be migrated to the cloud environment. After identification and assessment of each application an appropriate migration strategy is chosen for it.

Rehost strategy is also called as lift and shift method. In this method an application is lifted from on-premises servers and hosted in a cloud server. Enterprises that are looking to migrate quickly without disturbing existing applications will choose rehost strategy. Rehost removes the need for on-premises servers, large network and internet connection, and manpower to manage the servers. When an enterprise wants to migrate quickly at minimal cost the rehost approach is the ideal choice. Large-scale legacy systems are typically migrated using this approach.

The **replatform** approach is also called the lift-tinker-and-shift approach. In this approach codebase of an application will not be modified but its platform may be modified a little to reduce cost and improve performance. These modifications do not affect the functioning of an application. Consider a company that is maintaining records and data manually. However, modifications are introduced so that database management will be handled by Database as a Service cloud like Amazon Relational Database Service (Amazon RDS).

Legacy applications are generally not easy to maintain or showcase using modern approaches or platforms like mobile platforms. Refactoring involves legacy code being converted into fully maintainable code. The application is restructured to improve its performance, portability (without changing core functions), and external behavior. The critical or core application is preserved. **Refactoring** can be employed at the code level or architecture level so that the application can utilize cloud services.

In the **replace or re-purchase** approach the existing application is replaced with some commercial software, like COTS software, or with an application provided by the SaaS cloud provider. Software applications are offered in SaaS cloud. The applications are fully managed by SaaS cloud and data are maintained by users. If needed, an enterprise can migrate data to cloud-based storage.

If an application is found to be unsuitable for migration, it will have to be **retained** in the old environment. For example, an application that has many interdependencies is unsuitable for migration. Such applications are best retained in the on-premises setup and revisited later. They may be taken for decommissioning at a later time.

If an application is found not to be required during the discovery phase, then it should be chosen for the retire strategy. In some cases the application may not be required but will still be running and consuming resources. Hence such applications should be considered for the **retire** option.

----- SERVERLESS

Function as a Service (Faas) is an important approach to software development where you create small, specialized functions to handle specific tasks. Instead of building entire applications, developers focus on writing these functions, which are then executed in response to events or requests. Common

triggers include HTTP requests (e.g., AWS Lambda via API Gateway), File uploads (e.g., to Amazon S3), Database changes (e.g., DynamoDB streams) or Scheduled events.

Serverless is a cloud computing execution model where developers run applications or functions without managing servers or infrastructure. The cloud provider automatically handles server provisioning, scaling, maintenance, and availability, allowing developers to focus purely on writing code. Key Characteristics of Serverless:

- **No Server Management:** You don't manage operating systems, patching, or scaling. The provider runs your code on demand.
- **Event-Driven:** Serverless functions are triggered by events (e.g., HTTP requests, file uploads, database updates). *Ideal for event-driven tasks but not optimal for long-running, heavy workloads like video rendering or ML training.*
- **Auto-Scaling:** The platform automatically scales up when demand increases and scales down to zero when idle.
- **Pay-as-You-Go:** You pay **only for the compute time used**, not for idle server capacity.
- **Stateless Execution:** Each execution of a serverless function is **isolated** and doesn't retain state between runs (state must be stored externally).

Serverless does not mean "no servers." The name notwithstanding, servers in serverless computing are managed by a cloud service provider (CSP). Serverless describes the developer's experience with those servers - they are invisible to the developer, who doesn't see them, manage them or interact with them in any way. Serverless computing allocates machine resources on an as-used basis. Under a serverless model, developers can build and run applications without having to manage any servers and pay only for the exact amount of resources used. Instead, the cloud service provider is responsible for provisioning, managing, and scaling the cloud infrastructure that runs the application code.

Examples of Serverless Platforms: AWS Lambda (Amazon), Azure Functions (Microsoft), Google Cloud Functions, IBM Cloud Functions, Cloudflare Workers, etc

Tham khảo:

- <https://www.ibm.com/think/>
- <https://aws.amazon.com/>

CHƯƠNG V – Cloud Security

Several notions

- *Zero-trust architecture: Zero-trust architecture (ZTA) is a security model where nothing is trusted by default, even if it is inside the network perimeter. Every access request (from users, devices, or services) must be authenticated, authorized, and continuously verified, regardless of the source. This is especially important in cloud environments where traditional network boundaries are blurred.*
- *Data segregation: making sure one tenant's data cannot be accessed or leaked to another tenant due to misconfigurations, vulnerabilities, or attacks. Multi-tenancy means multiple customers (tenants) share the same physical resources (servers, storage, networks) in a cloud environment.*

DATA SECURITY

Identity Access Management

IAM (Identity and Access Management) is a framework of policies, tools, and technologies that ensures the right individuals and services have the right level of access to cloud resources. It handles:

- *Authentication – Verifying user identities (e.g., passwords, MFA).*
- *Authorization – Defining what resources users can access and what actions they can perform.*
- *User lifecycle management – Creating, updating, and removing accounts.*
- *Access control policies – Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC).*

Data Loss Prevention (DLP)

DLP refers to the processes and technologies used to *prevent sensitive data from being lost, misused, or accessed by unauthorized individuals*. It involves monitoring, detecting, and preventing risky data handling activities to protect confidential information. DLP strategies and tools help organizations meet regulatory compliance requirements, protect intellectual property, and improve data visibility.

Tokenization

Tokenization in cloud security replaces sensitive data with unique, non-sensitive tokens, significantly reducing the risk of data breaches and simplifying compliance. This process involves replacing sensitive data with tokens that have no inherent value, while the original data is stored securely in a vault. This approach allows for the continued use of data for various operations without directly exposing the sensitive information.

Federation *allows two or more organizations to share authentication and user identity information securely, while each company keeps its own internal directory (e.g., Active Directory, LDAP) private.* With federation, a trust relationship is established between the companies' identity providers (IdPs). When a user from Company A accesses Company B's application, Company B trusts Company A's authentication response. Identity Providers (IdPs) are services or systems that manage and verify digital identities (user credentials and attributes) within an organization.

INFRASTRUCTURE SECURITY

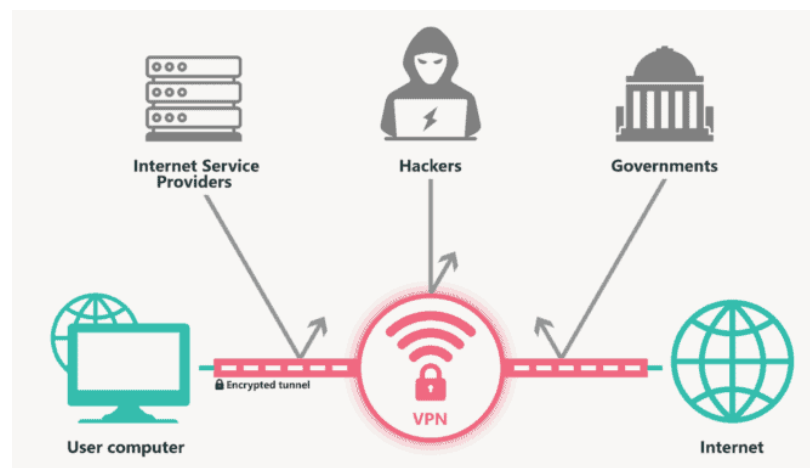
Network security protocols are standardized rules and procedures designed to *protect data during transmission over networks. They ensure confidentiality, integrity, authentication, and availability of data.* Several Network-Security Protocols

- **SSL/TLS (Secure Sockets Layer / Transport Layer Security):** SSL, or Secure Sockets Layer, is an encryption-based Internet security protocol. SSL is the direct predecessor of another protocol called TLS (Transport Layer Security). *TLS ensures that data is not altered or tampered with during transmission. TLS verifies the identity of the server using digital certificates (SSL/TLS certificates). This prevents attackers from impersonating the server (man-in-the-middle attacks).*
- **IPsec (Internet Protocol Security):** IPsec (Internet Protocol Security) is a suite of protocols that provide secure communication over an IP network by encrypting and/or authenticating each IP packet. It's commonly used to establish secure connections, especially in Virtual Private Networks (VPNs). IPsec ensures data confidentiality, integrity, and authenticity during transmission
- **HTTPS (HTTP Secure):** Hypertext transfer protocol secure (HTTPS) is the secure version of HTTP, which is the primary protocol used to send data between a web browser and a website. HTTPS is encrypted in order to increase security of data transfer. This is particularly important when users transmit sensitive data, such as by logging into a bank account, email service, or health insurance provider.
- **SSH (Secure Shell):** is a cryptographic network protocol that enables secure remote access and management of computers over an unsecured network. It operates on a client-server model, where an SSH client connects to an SSH server running on the remote machine.

Network segmentation divides a computer network into smaller parts. The purpose is to *improve network performance and security*. Other terms that often mean the same thing are network segregation, network partitioning, and network isolation. Segmentation improves cybersecurity by limiting how far an attack can spread. For example, segmentation keeps a malware outbreak in one section from affecting systems in another. Segmentation can stop harmful traffic from reaching devices that are unable to protect themselves from attack.

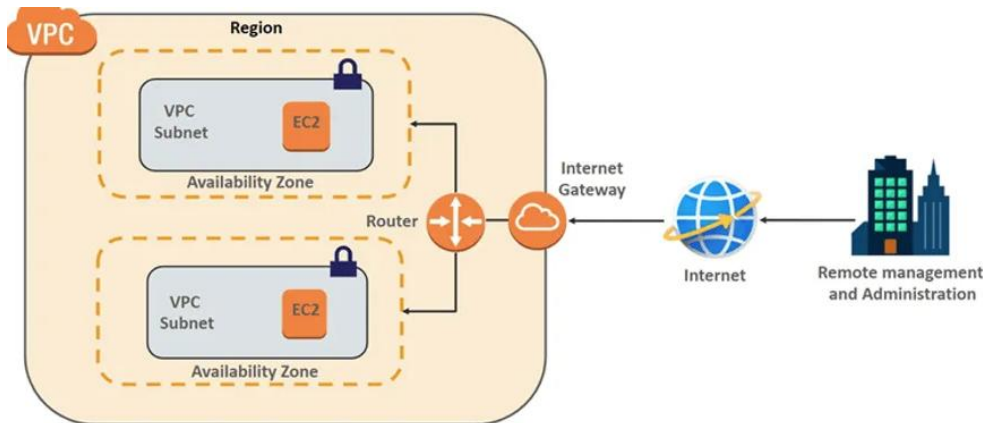
Virtual Private Network (VPN)

A VPN (Virtual Private Network) in cloud security is primarily used to create a secure, encrypted connection between a user's device or on-premises network and the cloud environment. It isolates traffic from the public internet, ensuring that sensitive data cannot be intercepted. VPNs enhance privacy, encryption, and secure remote access.



A VPC (Virtual Private Cloud) *enhances security in cloud environments by allowing organizations to create a logically isolated private network within a public cloud* (e.g., AWS, Azure, GCP). Key security features of a VPC include:

- **Secure VPN Connections:** A VPC can be connected securely to on-premises networks using IPsec-based VPNs, ensuring encrypted communication between the cloud and local data centers.
- **Network Segmentation:** VPCs allow the creation of private subnets, security groups, and network access control lists (NACLs) to control traffic flow.
- **Isolation:** Each VPC is isolated from other VPCs and cloud tenants, reducing the attack surface.

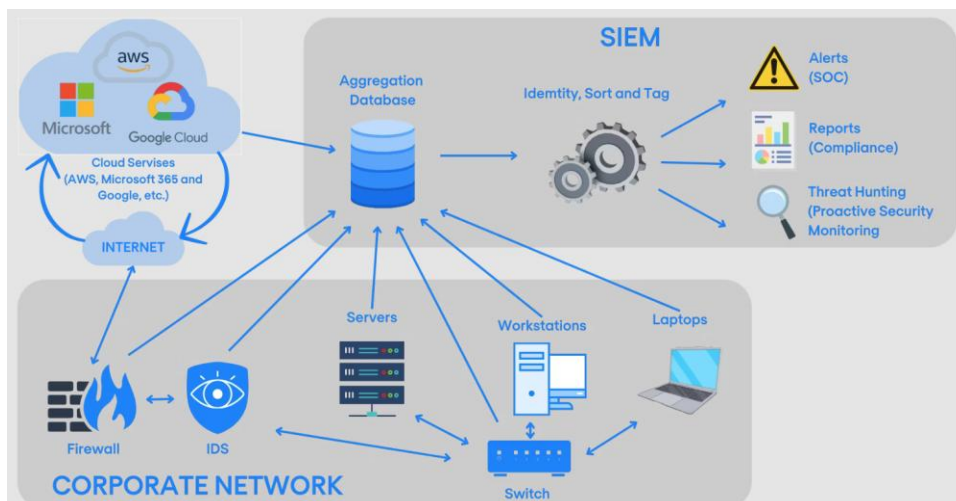


SIEM

Security information and event management, or SIEM, is a security solution that helps organizations recognize and address potential security threats and vulnerabilities before they have a chance to disrupt business operations.

SIEM systems help enterprise security teams detect user behavior anomalies and use artificial intelligence (AI) to automate many of the manual processes associated with threat detection and incident response.

The original SIEM platforms were log management tools. They combined security information management (SIM) and security event management (SEM) functions. These platforms enabled real-time monitoring and analysis of security-related events.



Tham khảo:

- <https://www.ibm.com/think/>
- <https://aws.amazon.com/>

CHƯƠNG IV – Cloud Native

Several notions

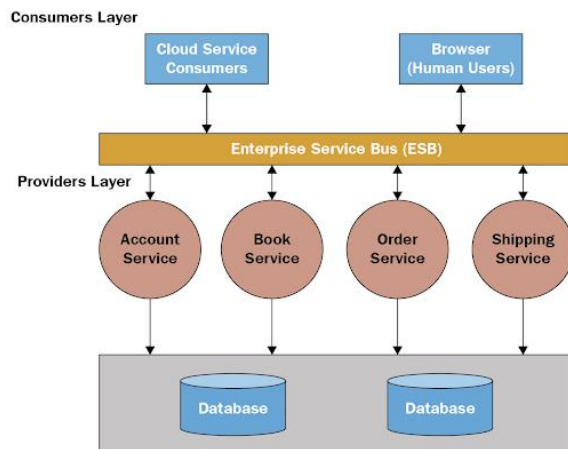
- Cloud native is an application development approach that uses technologies such as microservices, containers, container orchestrators and immutable infrastructure to build, deploy, run and manage responsive, scalable and fault-tolerant software applications in a cloud computing environment.
- Infrastructure as Code (IaC) is the practice of managing and provisioning cloud or on-premises infrastructure through code and automation rather than manual processes.* With IaC, servers, networks, databases, and other infrastructure components are defined in declarative or imperative scripts.

MICROSERVICES

Service-Oriented Architecture

Service-oriented architecture (SOA) designs software systems where different parts (or services) are used to build applications. Each service is designed to perform a specific job. These can be things like processing payments or verifying user identities. Each service communicates with each other across a network using formats and protocols like HTTP, REST, or SOAP to exchange information. The main goal of SOA is to make software development more flexible and modular by allowing these services to be developed and managed separately.

Each service in an SOA embodies the code and *data* required to run a complete, discrete business function (for example checking a customer's credit, calculating a monthly loan payment or processing a mortgage application). The service interfaces provide loose coupling. This means that they can be called with little or no knowledge of how the *service* is implemented underneath, reducing the dependencies between applications.



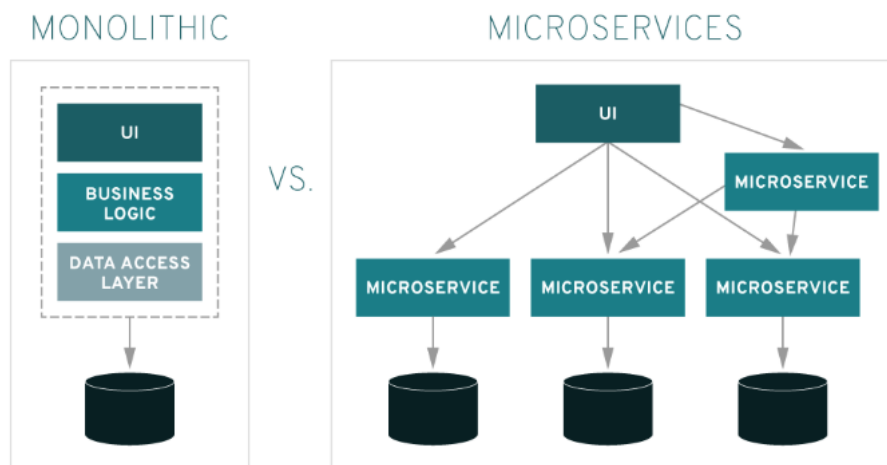
Microservices

In a microservices architecture, applications are composed of small, independent services. The services are loosely coupled, communicating via an API rather than via direct method calls, as in tightly coupled monolithic application. *A microservices architecture is more flexible than a monolithic application, because it allows you to independently scale, update, or even completely replace each part of the application.*

A dynamic microservices architecture *allows rapid scaling-up or scaling-down, as well as deployment of services across datacenters or across cloud platforms*. Resilience mechanisms provided by the cloud platform or built into the microservices themselves allow for self-healing systems. Finally, networks and datacenters become software defined, providing businesses with even more flexibility and agility for rapidly deploying applications and infrastructure.

A microservices architecture is a cloud-native approach to building software in a way that allows for each core function within an application to exist independently. When elements of an application are segregated in this way, it allows for development and operations teams to work in tandem without getting in the way of one another. This means more developers working on the same app, at the same time, which results in less time spent in development.

- Service registry: Stores where services are located
- Service discovery: The mechanism or process used by clients (or other services) to find and communicate with services by querying the service registry.



Tools and technologies that enable microservices

Containers and Kubernetes

A *Kubernetes is a container orchestration platform* that allows for single components within an application to be updated without affecting the rest of the technology stack, which makes it perfect for automating the management, scaling, and deployment of microservices applications.

APIs

An application programming interface, or API, is the part of an application that is responsible for communicating with other applications. Within the infrastructure of microservices architecture, APIs play the critical role of *allowing the different services within a microservice to share information and function as one*.

Event streaming

An event can be defined as anything that happens within a microservice service. For example, when someone adds something to their online shopping cart, or removes it. Events form into event streams, which reflect the changing behavior of a system. *Event stream processing allows immediate action to be taken, it can be used directly with operational workloads in real-time*.

Message broker

A *message broker* (e.g., RabbitMQ, Kafka, ActiveMQ) offers several benefits in microservices architecture:

- *Load Balancing and Scalability*: The broker can distribute messages among multiple consumers, enabling horizontal scaling.
- *Asynchronous Communication*: Services don't have to wait for a response; they can send messages and continue processing, improving performance and resilience.
- *Decoupling Services*: Senders don't need to know the details of receivers - senders just send messages to a message broker, without knowing which services will process those messages/ how many receivers exist/ what programming languages, APIs, or platforms the receivers use.

Serverless computing

Serverless computing is a *cloud-native development model that allows developers to build and run applications while a cloud provider is responsible for provisioning, maintaining, and scaling the server infrastructure*. Developers can simply package their code in containers for deployment. Serverless helps organizations innovate faster because the application is abstracted from the underlying infrastructure.

CONTAINERS AND KUBERNETES

Containers are executable units of software that package application code along with its libraries and dependencies. More portable and resource-efficient than virtual machines (VMs), containers have become the de facto compute units of modern cloud-native applications.

Containers depend on containerization, the *packaging of software code with just the operating system (OS) and its relevant environment variables, configuration files, libraries and software dependencies. The result is a container image that runs on a container platform. A container image represents binary data that encapsulates an application and all its software dependencies.*

Containerization allows applications to be “written once and run anywhere,” providing portability, speeding the development process, preventing cloud vendor lock-in and more.

One of disadvantage of using containers is Storage - Containers are temporary, meaning any data stored inside a container is lost when the container stops unless persistent storage is explicitly configured. Persistent storage is storage that retains data even after a container or application is stopped, restarted, or redeployed.

- Without persistent storage: Data written inside a container is stored in its writable layer, which is deleted when the container is removed or restarted.
- With persistent storage: Data is stored outside the container (e.g., in a volume, NFS, or cloud storage), ensuring that it remains intact regardless of the container's lifecycle.

Docker

Using Docker, you can quickly deploy and scale applications into any environment and know your code will run. You can do this because Docker packages software into standardized units called containers that have everything the software needs to run including libraries, system tools, code, and runtime.

Compared to Virtual machines (VMs) which virtualize server hardware while containers virtualize the operating system of a server. Docker is an operating system (or runtime) for containers. The Docker Engine is installed on each server you want to run containers on and provides a simple set of commands you can use to build, start, or stop containers.

The following are the differences between docker containers and Virtual Machines:

Docker Containers	Virtual Machines
Docker Containers contain binaries, libraries, and configuration files along with the application itself.	Virtual Machines (VMs) run on Hypervisors, which allow multiple Virtual Machines to run on a single machine along with its own operating system.
They don't contain a guest OS for each container and rely on the underlying OS kernel, which makes the containers lightweight.	Each VM has its own copy of an operating system along with the application and necessary binaries, which makes it significantly larger and it requires more resources.
<i>Containers share resources with other containers in the same host OS and provide OS-level process isolation.</i>	They provide Hardware-level process isolation and are slow to boot.

Container Orchestration

Orchestration is a methodology that provides a top-down view of your containers, giving you visibility and control over where containers are deployed and how workloads are allocated across multiple containers. Orchestration is essential to deploying multiple containers. Without orchestration, you must manage each container manually. Orchestration also allows IT managers to apply policies, such as fault tolerance, selectively or holistically to a collection of containers.

One of the enhanced capabilities afforded by container orchestration is the ability to automatically manage workloads across multiple compute nodes. (Nodes refer to any system connected to a network.) For example, if you have five servers but one server initiates a maintenance cycle, the orchestrator can automatically divert the workload to the four remaining servers and balance the workload based on what the remaining nodes can handle. The orchestrator can perform this task without human assistance.

CI/CD

Check slides – Cloud native.

DEVOPS

DevOps is a term that fuses “development” and “operations,” aiming to improve collaboration and communication between these two departments. It originated to address the shortcomings of traditional software development methods, particularly the gap between rapid code development and deployment. DevOps is not just a technology, but a philosophy that includes culture changes, such as building trust and aligning projects with business requirements.

It's particularly vital in today's fast-paced development cycles, enabling quicker time-to-market and continuous improvement. The main components of DevOps include people (the teams), processes (like CI/CD), and technology (tools for automation and monitoring).

Automation is one of the core principles of DevOps. It helps by: Eliminating repetitive manual tasks, reducing human error/ Speeding up the CI/CD pipeline (building, testing, and deploying faster)/Ensuring consistent and repeatable processes, leading to reliable software releases/ Allowing teams to release more frequently with higher quality.

Lifecycle Stages Explained:

- Plan: Define objectives, gather requirements, and roadmap projects.
- Develop: Write and review code using tools like Git.
- Build: Automate builds to integrate dependencies and prepare binaries.
- Test: Validate code with automated testing to ensure quality.
- Release: Use tools to manage release workflows securely and efficiently.
- Deploy: Automate the deployment process using tools like Kubernetes.

A test environment in DevOps is a setup that closely resembles the production environment where new code, features, and integrations are tested for functionality, performance, and bugs before deployment.

Key DevOps Tools

- Version Control: Git, GitHub, GitLab (track and manage code).
- CI/CD Pipelines: Jenkins, CircleCI (automate build and deployment).
- Configuration Management: Ansible, Puppet, Chef (manage system configurations).
- Containerization: Docker, Kubernetes (simplify deployment and scaling).
- Monitoring: Prometheus, Nagios, Splunk (ensure system health).

Tham khảo:

- <https://www.ibm.com/think/topics/docker>
- <https://aws.amazon.com/docker/>
- <https://www.geeksforgeeks.org/devops/introduction-to-docker/>
- <https://www.paloaltonetworks.com/cyberpedia/what-is-cloud-native>